Catching an FP16 Training Divergence with nixnan: From the First Large Value to the First NaN in TinyGPT

SC26 nixnan tutorial — `flash-attn-expt/fp16-divergence` (branch `Flash-Attn`)

October 2026

Abstract

We train TinyGPT, a 4-layer, 0.79M-parameter decoder-only transformer, entirely in IEEE half precision (FP16). Parameters, activations and gradients are all FP16; there are no FP32 master weights and no loss scaling. We use plain SGD with momentum at learning rate 1.0, which is high enough to make training diverge. The run is deterministic: the loss rises from 2.53 to 649 over nine steps and is NaN at step 10.

We monitor this run with nixnan, an NVBit-based binary instrumentation tool. Without changing the program, nixnan reports the values crossing 2^9, 2^{10} and 2^{15} at the FP16 residual addition in steps 7, 8 and 9, each attributed to a SASS instruction. In step 10 it reports $\pm\infty$ operands in that addition and then the first NaN, inside LayerNorm's $x - \mu$. A small watcher kills the run the moment nixnan prints that NaN. An independent PyTorch-side oracle identifies the same culprit: block 0's attention output projection, whose output reaches 65,440 and then ∞ (FP16's largest finite value is 65,504).

Getting there required four workarounds, and each one says something about the tool. Tile-based kernels (cuBLAS/CUTLASS GEMMs, flash attention, PyTorch's warp softmax) report false NaNs from padding lanes in a perfectly healthy first step. `MAX_ERRORS` cannot stop at a NaN because subnormals count too. nixnan's histogram mode crashes on large kernels because of an unbounded instruction table. And kernel names are truncated. We document each one, with logs.

Contents

1 Goal **3**

2 Background **3**
- 2.1 FP16 . . . 3
- 2.2 How PyTorch computes in FP16 . . . 3
- 2.3 nixnan . . . 3

3 The experiment **4**
- 3.1 TinyGPT . . . 4
- 3.2 FP16 training without safeguards . . . 4
- 3.3 The oracle . . . 4

4 What the oracle sees **5**

5 The nixnan setup **5**

6 Results **7**
- 6.1 The timeline . . . 7
- 6.2 Step 10, report by report . . . 7
- 6.3 The exceptions pass agrees . . . 8
- 6.4 Reproducibility . . . 8

7 Obstacles, and what they say about nixnan **9**
7.1 False NaNs in a healthy first step 9
7.2 `MAX_ERRORS` cannot stop at a NaN 9
7.3 The histogram crashes on large kernels 9
7.4 Truncated kernel names 10

8 Discussion **10**

9 Files and reproduction **10**

1 Goal

The request was: *make TinyGPT train in FP16 without loss scaling, at a learning rate high enough to diverge; then catch, with nixnan, the sequence of values that leads to ∞ and NaN; stop as soon as the NaN is seen.*

This is the textbook failure mode that mixed-precision training was designed to avoid. Real training recipes keep FP32 master weights and scale the loss dynamically (`torch.cuda.amp.GradScaler`), and they skip the optimizer step when gradients overflow. We remove all of that on purpose. The question is whether a binary-level tool, with no knowledge of the model, can show (1) *where* the first non-finite value appears, (2) *how* it propagates, and (3) whether the build-up is visible *before* anything overflows.

The short answer is yes on all three, with the caveats of Section 7.

2 Background

2.1 FP16

IEEE binary16 has 1 sign bit, 5 exponent bits and 10 fraction bits. Its largest finite value is $(2 - 2^{-10}) \cdot 2^{15} = 65{,}504$. Values at or above 65,520 round to ∞ in round-to-nearest. The smallest normal value is $2^{-14} \approx 6.1 \times 10^{-5}$, and subnormals go down to $2^{-24} \approx 6 \times 10^{-8}$. A binade 2^e means the magnitudes in $[2^e, 2^{e+1})$. The top binade of FP16 is 2^{15}, so a value seen in the 2^{15} binade is less than a factor of two from overflow.

2.2 How PyTorch computes in FP16

- **Elementwise kernels** (here `CUDAFunctor_add<c10::Half>`) load FP16, convert to FP32, compute in FP32 and convert back to FP16 on the store. `torch.add(a, b, alpha)` computes $a + \alpha b$, which compiles to one `FFMA` whose multiplier α is a kernel argument in constant bank 0 (`c[0x0][0x164]`).
- **LayerNorm** (`vectorized_layer_norm_kernel<c10::Half, float>`) accumulates the row mean and variance in FP32 and normalizes in FP32.
- **GEMMs** (`nn.Linear` in FP16) run on tensor cores (`HMMA.16816.F32`) with FP32 accumulation, then round the result to FP16. An accumulated value above 65,504 becomes $\pm\infty$ in that final conversion.

A consequence that matters for nixnan: in these kernels, most of the FP16 data is *seen as FP32 operands*. The FP16 histogram bins never fire in our run; the FP32 bins do.

2.3 nixnan

nixnan is an NVBit tool, loaded with `LD_PRELOAD`, that instruments every floating-point SASS instruction of the kernels it is allowed to touch. It needs no recompilation.

- **Exceptions mode** (default) checks the operands of each instrumented instruction and prints a report when it sees NaN, $\pm\infty$ or a subnormal:

```
#nixnan: error [NaN] detected in operand 0 of instruction FADD R0, R19, -R16 ; in function void at::
     native:: of type f32
```

  Operand 0 is the destination register, so a report on operand 0 means that the instruction *produced* the value; operands $1, 2, \ldots$ are the sources in SASS order.
- **Histogram mode** (`HISTOGRAM=1`, `BIN_SPEC_FILE`) counts the binades of the operands. The bin spec lists exponent ranges per format. A key of the form `"f32 (record_inst)"` also reports the instruction that caused each threshold hit. With `"count": 1` and `"doublings": N`, a bin reports at its 1st hit, then after 2, 4, 8, ... more, for N doublings, and then resets. This makes the first hit of a bin visible without flooding the log.
- **Filtering**: `FUNCTION_WHITELIST` / `FUNCTION_BLACKLIST` take a file of exact demangled kernel names. `MAX_ERRORS` stops the program after N exception reports. `LOGFILE` redirects nixnan's output.

3 The experiment

3.1 TinyGPT

TinyGPT (`../tinygpt.py`) is the model of the earlier flash-attention study: a pre-LayerNorm decoder over 12 tokens (digits 0–9, BOS, EOS), with $d_{\text{model}} = 128$, 4 heads of dimension 32, an MLP of width 512 with GELU, rotary position embeddings and no biases. It has 791,808 parameters. Each block computes

$$x \leftarrow x + W_{\text{proj}}\,\text{Attn}(\text{LN}_1(x)), \qquad x \leftarrow x + W_{\text{fc2}}\,\text{GELU}(W_{\text{fc1}}\,\text{LN}_2(x)).$$

Attention is PyTorch's SDPA with the flash backend forced. The training set is 100 digit strings, and the whole set is one batch: 100 sequences of 21 positions.

Two hard-coded `.float()` casts in `tinygpt.py` were changed to follow the weight dtype, so the model can run in FP16:

```
x = (idx[..., None] == torch.arange(VOCAB, device=idx.device)).to(self.emb.weight.dtype) @ self.emb.weight
y = flash_attention(q.half(), k.half(), v.half(), causal, self.idx).to(x.dtype)
```

The FP32 model is bit-for-bit unchanged: the completion of `312` is still `312452`.

3.2 FP16 training without safeguards

`train_fp16.py` (excerpt):

```
model = TinyGPT().cuda().half()                    # FP16 everything, no master weights
opt = torch.optim.SGD(model.parameters(), lr=a.lr, momentum=0.9)
for step in range(1, a.steps + 1):
    logits = model(x)
    loss = F.cross_entropy(logits.reshape(-1, logits.shape[-1]).float(), y.reshape(-1), ignore_index=-100)
    opt.zero_grad(set_to_none=True)
    loss.backward()
    lv = loss.item()
    print(f"step {step:4d}  loss {lv:.4f}", flush=True)
    if lv != lv or abs(lv) == float("inf"):
        print(f"STOP: non-finite loss at step {step}", flush=True)
        break
    opt.step()
```

The only FP32 computation is the cross-entropy on the logits, which are cast to FP32 so that the loss value itself is meaningful. There is no `GradScaler`, no gradient clipping and no master copy; `opt.step()` updates the FP16 parameters in place.

Why SGD at lr 1.0. SGD's update is $\Delta w = -\eta\,(\text{momentum-averaged gradient})$, so the step size grows with the gradient and the weights can grow without bound. (AdamW, used in `../train.py`, normalizes the update and would diverge far more slowly.) At $\eta = 1$ TinyGPT diverges in ten steps, which keeps the nixnan runs short.

Determinism. The seed is fixed and the whole data set is one batch. Every run we made, with or without nixnan, printed the same loss sequence to four decimals.

3.3 The oracle

`train_fp16.py -oracle` registers a forward hook on every leaf module and, after each backward pass, prints the largest finite $|\cdot|$ over activations, gradients and parameters, the tensor that holds the largest value, and the first tensor (in forward order, then gradients, then parameters) that contains a non-finite value. The oracle launches its own kernels, so it is never run under nixnan. It is the ground truth that nixnan's reports are checked against.

4 What the oracle sees

Table 1 is `logs/oracle-lr1.txt`. Figure 1 plots it.

step	loss	max |act|	max |grad|	max |param|	largest activation
1	2.5254	4.77	0.147	1.00	`blocks.1.ln2`
2	2.4189	4.66	0.109	1.005	`blocks.2.ln2`
3	3.1881	9.85	0.622	1.011	`head`
4	24.8335	24.3	2.23	1.015	`head`
5	6.7435	47.6	0.821	1.52	`head`
6	79.3333	87.0	6.27	3.23	`head`
7	169.8536	815.5	25.8	6.47	`blocks.0.proj`
8	198.7644	1,315	29.4	23.7	`blocks.0.proj`
9	649.3550	29,570	41.3	46.3	`blocks.0.proj`
10	NaN	65,440 + 7,870 ∞	NaN	66.1	`blocks.0.proj`

Table 1: Oracle, one row per training step (values after the forward and backward pass of that step). At step 10 the first non-finite tensor is the *output of* `blocks.0.proj`: 7,870 ∞ and no NaN.

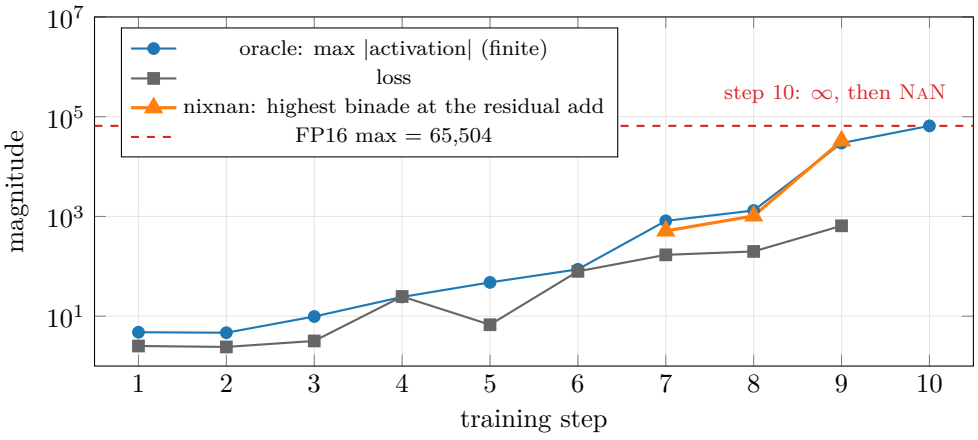


Figure 1: The climb. nixnan's histogram (orange; lower edge of the binade) tracks the oracle's activation maximum from step 7 on, three steps before the overflow.

What happens:

1. Steps 1–6: the weights grow (max $|w|$ goes from 1.0 to 3.2), and the logits (`head`) hold the largest activations. The loss oscillates and climbs.
2. Step 7: block 0's attention output projection takes over. Its output grows by a factor ≈ 1.6, then ≈ 22, then ≈ 2.2 per step: its weights grow (max $|w|$ reaches 46 at step 9) and so does its input, and the two multiply.
3. Step 10: the projection's FP32-accumulated outputs exceed 65,504 for 7,870 elements. They round to $\pm\infty$ when the GEMM writes FP16. The other elements stay finite (the largest is 65,440, two FP16 ulps of 32 below the maximum). The NaN in the loss comes from what happens next.

5 The nixnan setup

The final setup is `run_nixnan.sh`; Section 7 explains why it looks the way it does.

```
# pass 0: log every kernel name (dummy whitelist -> nothing is instrumented)
env LD_PRELOAD=$NN FUNCTION_WHITELIST=logs/dummy-whitelist.txt LOGFILE=logs/kernels-names.log \
    python train_fp16.py --lr 1
```

```
names | grep -E "pytorch_flash|cutlass|^ampere_|cublasLt|softmax_warp" > blacklist.txt
names | grep -E "CUDAFunctor_add<c10::Half>|vectorized_layer_norm_kernel<c10::Half" > hist-whitelist.txt

# pass 1: exceptions in every other kernel, killed at the first NaN report
python3 watch_nan.py logs/nixnan-exceptions.log -- \
    env LD_PRELOAD=$NN FUNCTION_BLACKLIST=blacklist.txt python train_fp16.py --lr 1

# pass 2: binade histogram with instruction attribution, two kernels, same watcher
python3 watch_nan.py logs/nixnan-climb.log -- \
    env LD_PRELOAD=$NN FUNCTION_WHITELIST=hist-whitelist.txt BIN_SPEC_FILE=spec-climb.json HISTOGRAM=1 \
    python train_fp16.py --lr 1
python3 summarize_climb.py logs/nixnan-climb.log logs/nixnan-exceptions.log > logs/SUMMARY.txt
```

Pass 0 (kernel names). One training run uses 50 distinct kernels. nixnan prints each kernel's full demangled name when it launches, even when nothing is instrumented. Whitelists and blacklists need exactly these names, so they are generated rather than written by hand.

Pass 1 (exceptions). 18 kernels are blacklisted (Table 2). All of them are tile-based and compute on lanes outside the tensor (Section 7.1). Every other kernel is checked for NaN, $\pm\infty$ and subnormals.

family	kernels	why
cuBLAS `ampere_fp16_s16816gemm_*`, `s1688gemm_*`	4	GEMM tiles
`cutlass::Kernel2<cutlass_80_*gemm*>`	6	GEMM tiles (NaN at step 1, `logs/false-positive-gemm-step1.log`)
`cublasLt::splitKreduce_kernel`	2	split-K reduction of a GEMM
`pytorch_flash::flash_*` (fwd, bwd, `dot_do_o`, `convert_dq`)	4	padding lanes (`../PyTorch_Instrumentation/PyT.md`)
`softmax_warp_forward/backward`	2	$-\infty$ padding (`logs/false-positive-softmax-step1.log`)

Table 2: `blacklist.txt`, generated in pass 0.

Blacklisting the GEMMs does not hide a GEMM overflow. An ∞ written by a GEMM is read by the next kernel, and nixnan sees it there as an ∞ source operand. That is exactly how the step-10 overflow shows up.

Pass 2 (histogram). `spec-climb.json`:

```
{ "count": 1, "doublings": 20,
  "f16 (record_inst)": [[9,9],[10,10],[11,11],[12,12],[13,13],[14,14],[15,15]],
  "f32 (record_inst)": [[9,9],[10,10],[11,11],[12,12],[13,13],[14,14],[15,15],[16,17]],
  "bf16": [], "f64": [] }
```

One bin per binade from 2^9 (≥ 512) to 2^{15} (the top FP16 binade), for FP16 and FP32 operands, plus one FP32 bin for 2^{16}–2^{17} (values FP16 cannot hold). Each bin reports its first hit, with the instruction. The whitelist holds the two kernels that touch the residual stream in FP16: the residual addition and LayerNorm. The histogram cannot cover every kernel (Section 7.3).

The watcher. `watch_nan.py` runs the command, prefixes every output line with the elapsed time, and on the first line that matches `#nixnan: error \[[^\]]*NaN` writes a `WATCHER` line and kills the whole process group with `SIGKILL`. Subnormal and $\pm\infty$ reports do not stop it.

Reading the log by step. `train_fp16.py` prints `step N loss X` after the backward pass of step N. Everything nixnan prints after the line for step $N-1$ therefore belongs to step N's forward and backward pass. `summarize_climb.py` uses this to group the reports by step and keeps, per kernel and format, the highest binade reached and the instructions that reached it.

6 Results

6.1 The timeline

Table 3 condenses `logs/SUMMARY.txt`.

step	loss	residual add (`CUDAFunctor_add`), FP32 operands	LayerNorm, FP32 operands	exception reports
1–3	2.53–3.19	below 2^9	below 2^9	none
4	24.8	below 2^9	2^{10} (`FADD`, `FFMA`)	none
5	6.74	below 2^9	2^{13}	none
6	79.3	below 2^9	2^{17} (`FMUL`: sums of squares)	none
7	169.9	2^9 ($\geq$ 512), `FFMA`; 3,040 reports	2^{17}	none (subnormals only)
8	198.8	2^{10} ($\geq$ 1,024); 1,292 reports	2^{17} (+`MUFU.RSQ`)	none
9	649.4	2^{15} ($\geq$ 32,768); 49,144 reports	2^{17}; 168,254 reports	none
10	—	2^{15}, then $\pm\infty$	$-\infty$, then NaN	add: $\pm\infty$ in `FFMA`; LayerNorm: $-\infty$, then **NaN** $\Rightarrow$ **killed**

Table 3: Per-step timeline from the histogram pass (columns 3–4) and both passes (column 5). "2^e" is the highest binade any instrumented instruction saw.

The residual add is the early warning. Its operands are the residual stream x and the block outputs. They cross 512 in step 7, 1,024 in step 8 and 32,768 in step 9, less than a factor of two from FP16's maximum. Every report names the instruction, always the same one: `FFMA Rd, Ra, c[0x0][0x164], Rc`, i.e. $d = \alpha\, b + a$. The oracle's maxima (815, 1,315, 29,570) fall in exactly these binades.

LayerNorm's large FP32 values are harmless. From step 4 on, LayerNorm's FP32 bins fire, and from step 6 they reach 2^{17}, a range FP16 cannot represent. These are the FP32 accumulations of $\sum(x-\mu)^2$ for the variance (`FMUL`, `FFMA`) and `MUFU.RSQ` of the variance. They are FP32 and never stored as FP16. A user reading the histogram must tell them apart from the residual stream, which is why the add is the useful signal.

The FP16 bins never fire. Both kernels convert to FP32 before computing (Section 2.2), so nixnan classifies every arithmetic operand as FP32. A bin spec with only `f16` keys would show nothing.

6.2 Step 10, report by report

The histogram pass printed these exception reports in step 10, in this order (bin reports omitted):

```
[54.0s] error [-infinity] operand 1 of FFMA R6, R7, c[0x0][0x164], R6   ; vectorized_elementwise_kernel (add)
[54.0s] error [-infinity] operand 0 of FFMA R6, R7, c[0x0][0x164], R6
[54.0s] error [infinity]  operand 1 of FFMA R9, R9, c[0x0][0x164], R8
[54.0s] error [infinity]  operand 0 of FFMA R9, R9, c[0x0][0x164], R8
[54.0s] error [infinity]  operand 1 of FFMA R10, R11, c[0x0][0x164], R10
[54.0s] error [infinity]  operand 0 of FFMA R10, R11, c[0x0][0x164], R10
[54.0s] error [-infinity] operand 1 of FADD R18, R19, -R16               ; void at::native:: (LayerNorm)
[54.0s] error [-infinity] operand 0 of FADD R18, R19, -R16
[54.0s] error [-infinity] operand 1 of FFMA R16, R18, R15, R16
[54.0s] error [-infinity] operand 0 of FFMA R16, R18, R15, R16
[54.0s] error [-infinity] operand 1 of FADD R0, R19, -R16
[54.0s] error [-infinity] operand 2 of FADD R0, R19, -R16
```

```
[54.0s] error [NaN]       operand 0 of FADD R0, R19, -R16
[54.0s] WATCHER: first nixnan NaN report -- killing the run
```

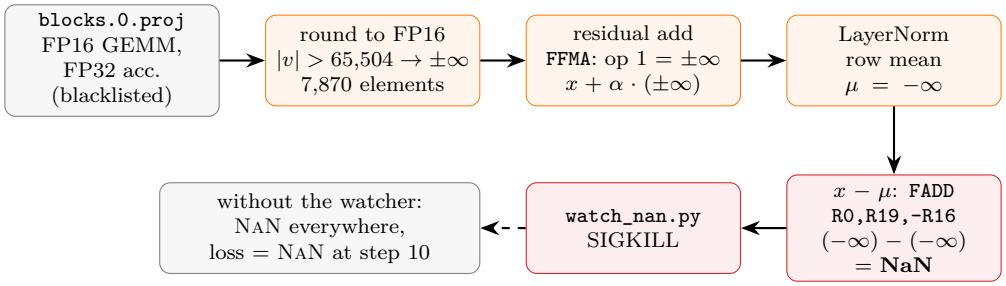


Figure 2: How the step-10 overflow becomes a NaN. Orange: $\pm\infty$; red: NaN.

Decoding them, with Figure 2:

1. **The residual add receives $\pm\infty$.** In `FFMA R9, R9, c[0x0][0x164], R8`, the sources are $R9 = b$ (the `other` tensor of `torch.add`), the constant α and $R8 = a$. For `x + self.proj(y)`, b is the projection's output. Operand 1 (b) is ∞, and so is operand 0 (the sum written back). The add did not create the ∞; it read it. Since the GEMMs are blacklisted, this is the first instrumented instruction to see the overflow, and the oracle confirms that the producer is `blocks.0.proj`.

2. **LayerNorm's mean becomes $-\infty$.** The next kernel to read the residual stream is block 0's `ln2`. `FADD R18, R19, -R16` followed by `FFMA R16, R18, R15, R16` has the shape of a running-mean update ($\delta = x - \mu$; $\mu \mathrel{+}= \delta \cdot \frac{1}{n}$), and the reports show an element $x = -\infty$ making δ and then μ equal to $-\infty$.

3. **The first NaN.** `FADD R0, R19, -R16` computes $x - \mu$ with x (operand 1) and μ (operand 2) both $-\infty$. The result (operand 0) is $(-\infty) - (-\infty) =$ NaN. This is the birth of the NaN, and the watcher kills the run in the same second.

Had the run continued, the NaN would have entered the variance, the normalized row, every following layer, the logits and the loss: the oracle's run shows loss = NaN and NaN gradients at step 10.

6.3 The exceptions pass agrees

Pass 1 instruments 32 kernels (all but the 18 blacklisted). It runs for 213 s and prints:

- no exception report in steps 1–7;
- 18 subnormal reports from step 8 on, in GELU's `FMUL`s (`0.3989\ldots` $= 1/\sqrt{2\pi}$);
- in step 10, the same six $\pm\infty$ reports in the add's three `FFMA`s, the same $-\infty$ reports in LayerNorm, and then the first NaN, killed.

In this pass the first NaN report is `operand 2 of FFMA R17, R18, R0, R17`, the instruction that consumes $R0 = x - \mu$, instead of the `FADD` that produced it. Both are in the same kernel, step and row computation. Which one prints first depends on which warp's report reaches nixnan's channel first; in our runs both orders occurred. It is always LayerNorm in step 10.

6.4 Reproducibility

We reran `run_nixnan.sh` from scratch (5 min 20 s). The loss sequence, the histogram timeline and the step-10 `FFMA`/`FADD` sequence were identical. The only difference was the first-NaN instruction described above.

7 Obstacles, and what they say about nixnan

7.1 False NaNs in a healthy first step

Our first attempt instrumented every kernel and used the watcher. It stopped at *step 1*, while the oracle shows every tensor of step 1 finite and bounded by 5.

CUTLASS GEMM. `logs/false-positive-gemm-step1.log`, 94 s into the run:

```
#nixnan: error [NaN,subnormal] detected in operand 1 of instruction HMMA.16816.F32 R20, R48.reuse, R46, R20 ;
    in function void cutlass::Kernel2 of type f16
```

A GEMM tile loads its A and B fragments from shared memory. When the problem does not fill the tile (for instance the 12-row vocabulary projection, far smaller than a tile), the out-of-range lanes compute on whatever bits are there, and only the in-range results are stored. The NaN is real in the register but never reaches a tensor.

Warp softmax. With GEMMs blacklisted, the next stop was `softmax_warp_forward` in `cross_entropy` (`logs/false-positive-softmax-step1.log`):

```
#nixnan: error [-infinity] detected in operand 0 of instruction FFMA R8, R7, 1.4426950216293334961, -R8 ; in
    function void  of type f32
#nixnan: error [NaN] detected in operand 0 of instruction FADD R7, -R12, R7 ; in function void  of type f32
```

PyTorch's warp softmax loads out-of-range elements as $-\infty$, so each warp can process whole rows of a power-of-two width. In a valid row, the padding elements give $(-\infty - \max) \cdot \log_2 e = -\infty$ (the first report; `1.4426\ldots` $= \log_2 e$) and $\exp(-\infty) = 0$, which is harmless. In a row slot past the last row, every element is $-\infty$, so $\max = -\infty$ and $x - \max = (-\infty) - (-\infty) =$ NaN (the second report). Such rows are never stored.

Same as flash. This is the padding-lane effect we documented for flash attention (`../PyTorch_Instrumentation/PyT.md`). For a tool that checks register operands, any tile-based kernel can report NaNs that are not in the program's data. Blacklisting these kernels is the practical answer, and it costs nothing here because their outputs are checked by the next kernel. A future nixnan could check stores (`INSTR_MEM`) instead of arithmetic for such kernels, or let users blacklist by pattern rather than exact name.

7.2 `MAX\_ERRORS` cannot stop at a NaN

`MAX_ERRORS=1` stops after the first exception report of any kind, and subnormals count. With GEMMs instrumented, FP16 training reports subnormal `HMMA` operands at step 1 (34 s in), long before any overflow. Even with GEMMs blacklisted, GELU's subnormals come at step 8, two steps before the NaN. We need "stop at the first NaN", which nixnan does not offer, so `watch_nan.py` supplies it from outside. A `MAX_NANS` (or a category mask for `MAX_ERRORS`) would make it unnecessary.

7.3 The histogram crashes on large kernels

With `HISTOGRAM=1` and no whitelist, the run segfaults (`logs/histogram-crash.log` ends abruptly at 39 s). Under gdb:

```
Thread 1 "python" received signal SIGSEGV, Segmentation fault.
... std::__cxx11::basic_string<...>::_M_assign(...)
#3  nixnan::recorder::mk_entry (this=..., instr="FFMA R12, R10, R2.reuse, R13 ;", path="", line="0",
      func="void cutlass::Kernel2", ...)
```

`recorder::mk_entry` (`src/recording.cu:88`) is:

```
inst_info[current_entry].instr = instr;
inst_info[current_entry].path  = path;
...
return current_entry++;
```

with `inst_info` allocated for `TABLE_SIZE` $= 2^{17}$ entries (`src/nixnan.cu:71`) and no bounds check. With `record_inst` keys, every FP instruction of every instrumented kernel gets an entry. The large GEMM kernels push `current_entry` past the end, the string assignments write into freed or foreign heap memory, and the process dies. We did not modify nixnan; restricting the histogram to two kernels keeps the table small. The fix is a bounds check (or a growable table), plus deduplication of entries per (function, offset).

7.4 Truncated kernel names

nixnan's `cut_kernel_name` keeps the demangled name up to the first `<` or `(`:

```
size_t pos = kernel_name.find_first_of("<(");
```

So `void at::native::(anonymous namespace)::vectorized_layer_norm_kernel<c10::Half, float>(...)` prints as `void at::native::`, and the warp softmax prints as `void` . Every elementwise kernel prints as `void at::native::vectorized_elementwise_kernel`. In the exception reports, LayerNorm is identifiable only because the histogram pass instruments just two kernels (the bin reports, which come from another code path, carry the full name). Cutting at the template list *after* the function name, or printing the full name once per kernel with a short id, would fix this.

8 Discussion

What nixnan showed that the loss curve does not. The loss curve says the run is unstable from step 4 and dead at step 10. nixnan says, without source changes: which instruction in which kernel first saw an ∞ (the residual add's `FFMA`, operand 1, i.e. an overflowed GEMM output), which instruction first produced a NaN (LayerNorm's $x - \mu$), the arithmetic reason ($-\infty - (-\infty)$), and three steps of warning in which the residual stream climbs binade by binade towards 65,504.

What it took. It took a model of the program to aim the tool: which kernels are tile-based (blacklist), which kernels carry the residual stream (histogram whitelist), and that FP16 data is computed on as FP32 (bin spec). It took a watcher for "stop at the first NaN". And the oracle was needed to name the blacklisted producer (`blocks.0.proj`), which nixnan can only see indirectly.

What a fix would look like. For the training run itself, the standard fixes each address one link of Figure 2: a smaller learning rate or AdamW (no growth), gradient clipping, FP32 master weights with `GradScaler` (overflow detected and the step skipped), or BF16 (8 exponent bits: the same values would be finite). nixnan's histogram would show the first two as a residual stream that never climbs.

9 Files and reproduction

All under `flash-attn-expt/fp16-divergence/`:

`train_fp16.py`	FP16 training; stops at the first non-finite loss; `-oracle`
`watch_nan.py`	runs a command, timestamps its output, kills it at the first nixnan NaN report
`run_nixnan.sh`	passes 0–2, summary, oracle (≈5 min on an RTX 3090)
`blacklist.txt`, `hist-whitelist.txt`, `spec-climb.json`	nixnan configuration
`summarize_climb.py`	per-step timeline from the logs
`logs/SUMMARY.txt`	the timeline (Table 3)
`logs/nixnan-exceptions.log`	pass 1
`logs/nixnan-climb.log.xz`	pass 2 (398k lines)
`logs/oracle-lr1.txt`	Table 1
`logs/false-positive-*-step1.log`	Section 7.1
`logs/histogram-crash.log`	Section 7.3
`logs/kernels-names.log`	pass 0
`latex/`	this report and the slides

Environment: PyTorch 2.6.0+cu124 (venv `~/repos/newmed/venv`), CUDA 12.4, RTX 3090 (sm_86), nixnan from `external/nixnan` (NVBit 1.8), unmodified.